

Firmware language: C/C++ via Arduino framework

The Arduino Nano's ATmega328P has only 32KB of flash and 2KB of RAM. that rules out anything like MicroPython or CircuitPython, which need far more headroom. Standard Arduino C/C++ is the only realistic choice for the flight side, and it's genuinely the right one: every library you need (BMP388, MPU6050, GPS/NMEA parsing, RFM95W, SD) already has mature Arduino support.

One upgrade worth considering: develop in **PlatformIO** (as a VS Code extension) instead of the plain Arduino IDE. Same language, same libraries, but much better project structure, dependency management, and version control friendliness — worth it if your brother's used to a real IDE workflow.

Firmware architecture: a non-blocking state machine

This is the part that actually matters most for reliability. Don't write this as one long `loop()` that reads sensors in sequence with `delay()` calls — a rocket flight has genuinely different phases, and the firmware should know which one it's in:

State	Trigger	Behavior
Pre-launch / Armed	Power-on	Idle sampling, waiting for launch detection (sudden acceleration spike)
Boost	Acceleration exceeds a threshold	Fastest sampling rate, log everything
Coast	Acceleration drops back near 0/1g	Continue logging, watch barometer for apogee (altitude no longer increasing)
Apogee / Deploy	Altitude peak detected	Trigger any deployment logic, mark event in log
Descent	Altitude decreasing	Lower sampling rate is fine here (battery/log-space conservation), GPS is now reliable again per our research
Landed	Altitude stable for N seconds	Stop high-rate logging, keep GPS/radio alive for recovery beacon

Use `millis()`-based timing instead of `delay()` throughout, so the barometer, IMU, GPS, radio, and SD card can all be serviced in the same loop without blocking each other — this matters a lot once you're juggling five peripherals on one small chip.

Sensor fusion: keep it simple — a complementary filter

Given the MCU's limited horsepower, a full Kalman filter is overkill for the accuracy gain it'd give you here. A **complementary filter** (blend barometer altitude with double-integrated accelerometer data, weighted to trust the barometer long-term and the accelerometer short-term) is lightweight, well-documented, and standard in hobby flight computers for exactly this reason.

Telemetry packets: compact binary, not text

LoRa has real but limited bandwidth. Send a fixed-size binary struct (timestamp, altitude, velocity, acceleration, lat/lon, state) rather than a human-readable string like

`"alt=1200,vel=45"` — the binary version is a fraction of the size, which matters for update rate and range margin.

Ground station: this is where your brother's strengths come in

The Nano itself has no display, so the "ground station" is really a second system. Given the plan already calls for a laptop-side receiver, here's a split that plays to both of your strengths:

- **Ground-side radio bridge:** a second cheap Arduino (or even reusing a spare Nano) wired to the second RFM95W, doing nothing but forwarding received LoRa packets out over USB serial as raw bytes. Minimal, boring, reliable firmware.
- **Ground station software:** a **Python app on a laptop** reading that serial port — this is where a strong software developer can genuinely add a lot of value beyond what the embedded side can do. A live dashboard (altitude/velocity graphs updating in real time, current GPS position optionally plotted on a simple map, a big "STATE: DESCENT" indicator) is very achievable with something like PyQt, Dear PyGui, or even a lightweight local web dashboard (Flask + a live-updating chart via WebSocket) if he'd rather build it as a webpage.